SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5116

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Assessment Tool Weightage**:** 10%

QUESTIONS: **Total Marks: 50**

Annexure A

Debugging & Optimization Task

Code of Conduct:

*I**M.SADHANA[192524386] (**Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

Debugging & Optimisation Task

1.Debugging & Optimization of Symmetric Encryption(DES / 3DES Implementation)
A Python program implementing **DES and 3DES encryption** produces inconsistent ciphertext for identical inputs across multiple runs.

A. Identify and fix logical and implementation errors in key scheduling and permutation steps.
B. Optimize the code to reduce execution time for large input datasets (≥10 MB).
C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

2.Performance Analysis of Block Cipher Modes of Operation A Python script implements **AES-like block cipher modes (ECB, CBC, CFB, OFB)** but suffers from high latency and memory inefficiency.

D. Debug incorrect padding and IV handling causing decryption failures.
E. Refactor the code to minimize redundant computations and memory usage.
F. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

3.Debugging RSA Cryptosystem Implementation A Python-based **RSA encryption system** fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

G. Identify issues in prime generation, key generation, and modular exponentiation.
H. Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).
I. Evaluate the trade-offs between key size, security, and computational performance.

4.Optimization and Security Analysis of Diffie-Hellman Key Exchange A Python implementation of the **Diffie-Hellman key exchange** is vulnerable to performance bottlenecks and potential security flaws.

J. Debug incorrect shared key generation due to improper modular arithmetic.
K. Optimize exponentiation operations for large prime values.
L. Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

5.Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation A Python script implementing **Elliptic Curve Cryptography** for key exchange produces incorrect results for certain curve parameters.

M. Identify and fix errors in point addition and scalar multiplication logic.
N. Optimize the implementation using efficient algorithms (e.g., double-and-add)

Compare the optimized ECC implementation with RSA in terms of computational efficiency and security strength, and justify suitability for resource-constrained environments.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

4. Optimization and Security Analysis of Diffie-Hellman Key Exchange

Introduction

Diffie-Hellman (DH) is a public-key key exchange algorithm that allows two users to generate a common secret key over an unsecured network. The secret key can then be used with a symmetric encryption algorithm such as AES to protect data. However, an incorrect implementation may produce different shared keys, slow performance, or security vulnerabilities.

A. Debug Incorrect Shared Key Generation Due to Improper Modular Arithmetic

The shared key must be calculated correctly using modular exponentiation.

Steps involved:

1. Select a large prime number (p) and a generator (g).
2. Alice chooses a private key (a), and Bob chooses a private key (b).
3. Alice calculates her public key:
4. (A = g^a \bmod p)
5. Bob calculates his public key:
6. (B = g^b \bmod p)
7. Alice calculates the shared secret:
8. (K = B^a \bmod p)
9. Bob calculates the shared secret:
10. (K = A^b \bmod p)
11. Both users obtain the same shared secret because:
12. (K = g^{ab} \bmod p)
13.

Common implementation errors:

Using ordinary division instead of the modulo operator %.

Applying modulo at the wrong stage of the calculation.

Generating weak or invalid private keys.

Using different values of (p) or (g) on the two sides.

Converting large numbers incorrectly or using an inefficient exponentiation method.

Correct Python example:

```
# Diffie-Hellman Key Exchange
p = 23
g = 5
# Private keys
a = 6
b = 15
# Public keys using modular exponentiation
A = pow(g, a, p)
B = pow(g, b, p)
# Shared secret key
key_alice = pow(B, a, p)
key_bob = pow(A, b, p)
print("Alice Public Key:", A)
print("Bob Public Key:", B)
print("Alice Shared Key:", key_alice)
print("Bob Shared Key:", key_bob)
if key_alice == key_bob:
    print("Key Exchange Successful")
```

```
else:
    print("Key Exchange Failed")
```

Output:

```
Alice Public Key: 8
Bob Public Key: 19
Alice Shared Key: 2
Bob Shared Key: 2
Key Exchange Successful
```

This small example is only for learning. Real applications must use approved cryptographic parameters and secure libraries.

B. Optimize Exponentiation Operations for Large Prime Values

Diffie-Hellman uses modular exponentiation, which can become computationally expensive when large prime numbers and private exponents are used.

Optimization techniques:

Fast modular exponentiation: Python's pow(base, exponent, modulus) calculates modular powers efficiently without first constructing the complete, extremely large power.

Avoid repeated calculations: Calculate each public key and shared secret only when needed.

Use secure cryptographic libraries: Established libraries provide optimized and tested implementations.

Choose approved parameters: Suitable prime sizes and groups provide security while avoiding unnecessarily expensive calculations.

Use efficient key management: Reuse only what the chosen protocol safely permits; private keys must be generated securely and protected.

For example, use pow(g, a, p) instead of pow(g, a) % p. The first method performs modular exponentiation efficiently and uses less memory for large calculations.

These improvements reduce execution time and memory usage while maintaining correct shared-key generation.

C. Mitigate Man-in-the-Middle Attacks and Evaluate Performance Impact

A major security weakness of basic Diffie-Hellman is its vulnerability to a **Man-in-the-Middle (MITM) attack**. An attacker may intercept the public keys exchanged between Alice and Bob and substitute their own keys. This can allow the attacker to establish separate shared secrets with both users.

Security enhancements:

Authentication: Verify the identity of each communicating party using digital certificates or digital signatures.

TLS protocol: Use a correctly configured Transport Layer Security (TLS) protocol to authenticate parties and protect communication.

Approved key exchange: Use standardized Diffie-Hellman or elliptic-curve Diffie-Hellman methods through trusted cryptographic libraries.

Secure key derivation: Apply a suitable key derivation function (KDF) to derive encryption keys from the shared secret.

Strong private keys: Generate private keys securely and never transmit them over the network.

Integrity and encryption: Use authenticated encryption, such as AES-GCM, to protect exchanged data from unauthorized modification and disclosure.

Performance impact:

Authentication and certificate verification add some computational overhead and may increase connection setup time. However, optimized modular exponentiation and efficient cryptographic libraries help reduce the delay. These additional security checks are important because faster communication is not useful if the connection can be intercepted or impersonated.

conclusion

Diffie-Hellman is an important method for securely establishing a shared secret over an unsecured network. Correct modular arithmetic prevents shared-key calculation errors, while fast modular exponentiation improves performance for large prime values. Authentication, digital certificates, and secure protocols help prevent Man-in-the-Middle attacks. Therefore, combining an optimized implementation with suitable security measures improves both the efficiency and reliability of Diffie-Hellman key exchange.

References

- NIST, *Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography*, SP 800-56A.
- IETF, *The Transport Layer Security (TLS) Protocol Version 1.3*, RFC 8446.
- Python Software Foundation, *Python Documentation: Built-in Functions — pow()*.